

Universidade Estadual de Campinas - UNICAMP
Instituto de Computação
Introdução ao Processamento de Imagem Digital (MC920 / MO443)

Trabalho 4

Aluno: Pedro Brasil Barroso - RA 260637

Professor: Dr. Hélio Pedrini

Campinas
1º Semestre de 2025

Sumário

1	Introdução	2
2	Estrutura do trabalho	2
2.1	Requisitos	2
2.2	Organização dos arquivos	2
2.3	Como executar os exercícios	3
3	Medidas de Objetos	4
3.1	Introdução	4
3.2	Metodologia	4
3.2.1	Pré-processamento	4
3.2.2	Cálculo da área	5
3.2.3	Cálculo do perímetro	6
3.2.4	Cálculo da solidez	6
3.2.5	Cálculo da excentricidade	6
3.3	Resultados	8
3.4	Limitações	10
3.5	Conclusão	10
4	Tranformações Geométricas	11
4.1	Introdução	11
4.2	Metodologia	11
4.2.1	Transformações espaciais	11
4.2.2	Forma de mapeamento	12
4.2.3	Métodos de interpolação	12
4.3	Resultados	13
4.4	Limitações	14
4.5	Conclusão	14

1 Introdução

O objetivo deste trabalho é aplicar e analisar algoritmos de representação, descrição e registro de imagens, explorando suas possibilidades de implementação, suas limitações práticas e os fundamentos teóricos nos quais se baseiam.

2 Estrutura do trabalho

2.1 Requisitos

As versões de Python e das bibliotecas utilizadas no projeto são as que seguem:

- Python: 3.12.3
- NumPy: 2.2.5
- OpenCV: 4.11.0.86
- Matplotlib: 3.10.1

2.2 Organização dos arquivos

O arquivo `.zip` submetido apresenta a seguinte organização (dentro do diretório `t4/`):

- `images/`
Contém as imagens de entrada utilizadas nos exercícios.
- `results/`
Armazena as imagens processadas (resultados).
- `helper_functions.py`
Script com funções auxiliares, as quais permitem salvar e exibir imagens, dentre outras funcionalidades.
- `measurement_methods.py` e `transformation_methods.py`
Scripts com definições de funções para calcular as medidas de descrição dos objetos e para realizar as transformações geométricas, respectivamente.
- `1_run.py` e `2_run.py`
Scripts que executam os exercícios 1 e 2, respectivamente.
- `requirements.txt`
Lista de dependências do projeto (bibliotecas Python necessárias).

2.3 Como executar os exercícios

Os scripts `1_run.py` e `2_run.py` são responsáveis por executar os algoritmos implementados neste trabalho. A execução deve ser feita via terminal, com os seguintes argumentos:

- Flags comuns a ambos os arquivos
 - `-i`: Nome da imagem (com extensão) localizada no diretório `images` a ser processadas. Se não especificado, uma imagem padrão é utilizada.
 - `-s`: Flag para salvar as imagens resultantes no diretório `results`. Os arquivos são nomeados como `exercicio_<nome>.png`.
Exemplos: `01/binaria.png`, `02/bicubica.png`,
 - `-d`: Flag para exibir as imagens processadas. É possível pressionar “q” para encerrar a exibição.
- Flags exclusivas de `2_run.py`
 - `-a`: Ângulo de rotação (em graus) no sentido anti-horário. Padrão: 0.
 - `-e`: Fator de escala. Padrão: 1.0.
 - `-D`: Dimensão da imagem de saída em pixels. A imagem resultante é centralizada no ponto médio da imagem. Por padrão, as dimensões são aquelas que permitem a visualização de toda a imagem.
 - `-m`: Métodos de interpolação utilizados. Os valores possíveis são: `nn` (vizinhos mais próximos), `bilinear`, `bicubic` e `lagrange`. Por padrão, todos são utilizados.

Exemplos de uso:

- Exibe e salva a imagem no exercício 1:

```
python3 1_run.py -i objetos1.png -d -s
```

- No exercício 2, rotaciona a imagem em 73°, com fator de escala 3.7, utilizando interpolação bicúbica e por vizinhos mais próximos. Também limita as imagens de saída a 128x128 e as exibe.

```
python3 2_run.py -i baboon_colorida.png -a 73.0 -e 3.7 \
-d -m nn bicubic -D 128 128
```

3 Medidas de Objetos

3.1 Introdução

A análise quantitativa de objetos em imagens binárias é uma etapa essencial em diversas aplicações de processamento de imagens e visão computacional. Métricas como área, perímetro, solidez e excentricidade permitem descrever geometricamente os objetos segmentados e servem como base para etapas posteriores de processamento.

Seja $\mathcal{O}_l$ um objeto da imagem associado a um rótulo l , resultante de um processo de rotulação de componentes conectados. Podemos representar a presença do objeto em uma imagem $\mathbf{f}(x, y)$ pela função $f_l(x, y)$, definida como:

$$f_l(x, y) = \begin{cases} 1, & \text{se } (x, y) \in \mathcal{O}_l \\ 0, & \text{caso contrário} \end{cases} \quad (1)$$

Com essa definição, a área do objeto l corresponde à contagem do número de pixels que o compõem, ou seja:

$$\text{Área}(l) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f_l(x, y) \quad (2)$$

O perímetro é definido como o comprimento da curva que delimita a borda do objeto. Ele pode ser por meio da soma das distâncias entre pontos adjacentes do contorno do objeto.

A solidez é uma medida dada pela razão entre a área do objeto e a área de seu fecho convexo (menor polígono convexo que o contém). Essa métrica fornece uma noção da densidade do objeto, sendo igual a 1 para objetos perfeitamente convexos e menor que 1 para objetos cuja forma apresenta concavidades.

$$\text{Solidez}(l) = \frac{\text{Área}(l)}{\text{Área do Fecho Convexo}(l)} \quad (3)$$

Por fim, a excentricidade quantifica o alongamento de um objeto, sendo baseada sua definição na geometria das elipses, a partir de seus eixos maior e menor, a e b , respectivamente:

$$e = \sqrt{1 - \frac{b^2}{a^2}}, \quad \text{com } a \geq b \quad (4)$$

Essa medida varia de 0 (quando a elipse é um círculo, $a = b$) até valores próximos de 1 (quando o objeto é alongado).

3.2 Metodologia

3.2.1 Pré-processamento

A etapa de pré-processamento foi realizada, inicialmente, a partir das seguintes etapas:

1. Binarização da imagem

Nesta etapa, foi assumido que a imagem processada possui fundo branco, i.e., com todos os canais R, G e B apresentando valores iguais a 255. Com base nisso, foi possível obter uma imagem binária, onde os objetos eram dados pelo valor 0 e o fundo pelo valor 1.

Constatou-se posteriormente, porém, que tal escolha resultou em comportamentos indesejados em outras etapas do processo, e foi alterada para que o fundo tivesse valor 0, e os objetos, 1.

2. Obtenção das bordas

Por definição, a borda de um objeto é dada pelos pontos os quais a ele pertencem e possuem menos que quatro vizinhos em vizinhança-4; para realizar esta etapa, foi realizada uma convolução da imagem pela seguinte máscara:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Note que, uma vez que a imagem possui pixels apenas com os valores 0 ou 1, os pontos de borda devem apresentar valores menores que 4 após a convolução. Contudo, como citado no item anterior, inicialmente foi atribuído aos pixels do fundo o valor 1; portanto, essa operação detectava as bordas com base no fundo, e não nos objetos. Assim, detectava-se as “bordas das bordas” dos objetos, e, uma vez que a estratégia utilizada na convolução foi de *padding* com o valor 0, as bordas da imagem também eram detectadas como bordas nessa etapa.

3. Rotulação das componentes conexas

A biblioteca *OpenCV* implementa esse processo através da função `connectedComponents`. Ela recebe uma imagem binária, na qual o fundo deve possuir o valor 0, e retorna uma máscara $\mathbf{f}'$, cujos pixels possuem valor dado por¹ $f'(x, y) = \sum_{l=0}^L l f_l(x, y)$, onde L é a quantidade de componentes encontradas pela função. Além disso, é atribuído ao fundo o rótulo 0.

Novamente, devido à representação escolhida na primeira implementação, o fundo foi detectado como um objeto da imagem, i.e., foi atribuído a essa região um rótulo diferente de 0.

3.2.2 Cálculo da área

O cálculo da área, após a rotulação das componentes, se torna um processo trivial, podendo essa medida ser obtida a partir da seguinte relação:

$$\text{Área}(l) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \begin{cases} 1, & \text{se } f'(x, y) = l \\ 0, & \text{caso contrário} \end{cases}$$

¹Uma definição alternativa: na máscara $\mathbf{f}'$, os pixels pertencentes a um objeto cujo rótulo é l apresentam valor igual a l .

3.2.3 Cálculo do perímetro

A biblioteca *OpenCV* possui duas funções que simplificam o cálculo do perímetro: `findContours` e `arcLength`. A primeira encontra um conjunto de pontos capazes de descrever a forma do objeto por um polígono - côncavo ou convexo, a depender da forma do objeto -, enquanto a segunda recebe um conjunto de pontos, e calcula a extensão das retas que os ligam. Assim, o perímetro foi calculado a como a extensão do contorno do objeto.

3.2.4 Cálculo da solidez

Para realizar o cálculo da solidez, foi necessário primeiro encontrar o fecho convexo da forma do objeto de análise. Isso foi realizado, novamente, por uma função implementada pela biblioteca *OpenCV*: `convexHull`, a qual calcula os pontos do fecho convexo. Após a obtenção dessa forma, sua área é calculada da mesma forma que a área do objeto; assim, para obter a solidez, é feita a divisão da área do objeto por essa.

3.2.5 Cálculo da excentricidade

O cálculo da excentricidade foi realizado por meio de dois métodos diferentes. O primeiro tem como base encontrar os eixos maior e menor a partir dos pontos do objeto, enquanto o segundo encontra uma elipse que descreve suficientemente bem o formato do objeto, utilizando os eixos dessa no cálculo.

Para a implementação da primeira, foi utilizada a seguinte definição: o eixo maior é o comprimento do maior segmento de reta interno ao objeto, ao passo que o eixo menor é o comprimento do maior segmento de reta perpendicular ao eixo maior, também interno ao objeto.

Assim, para encontrar o eixo maior, são calculadas as distâncias entre todos os pares de pontos do contorno do objeto, de forma que o par cuja reta interna que liga seus pontos possui comprimento máximo (igual à distância entre os pontos) é selecionado.

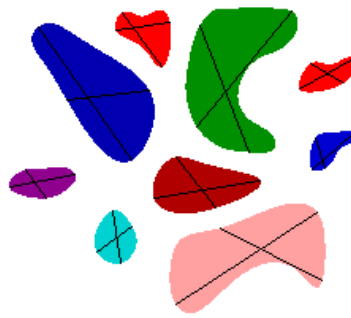
Em seguida, para calcular o eixo menor, utilizou-se inicialmente do fato que o produto interno entre dois vetores perpendiculares tem como resultado 0. Assim, novamente a partir da iteração sobre os pares de pontos, calculava-se o produto interno entre a reta que liga os pontos (desde que essa fosse interna) e a reta do eixo maior.

Contudo, devido à utilização das posições discretas da imagem para esse cálculo, foi necessário atribuir um limiar próximo de 0 de modo que, se o resultado do produto interno fosse menor que esse valor, as retas seriam consideradas perpendiculares.

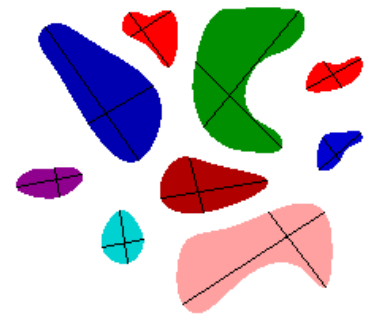
O processo, embora tenha produzido bons resultados inicialmente, se mostrou extremamente sensível à escolha de limiar e a transformações de escala. Assim, o processamento com um valor fixo de limiar não garantia resultados satisfatórios para todas as imagens utilizadas como entrada. Frente à sensibilidade à escala, não foi possível realizar um processo manual de *fine-tuning* do limiar, mostrando-se necessária a implementação de outra solução.



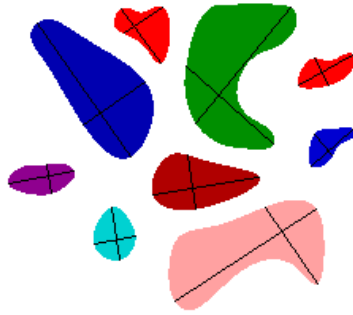
(a) Primeira implementação
(Limiar = 1)



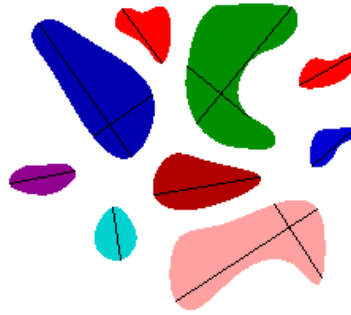
(b) Primeira implementação
(Limiar = 0.5)



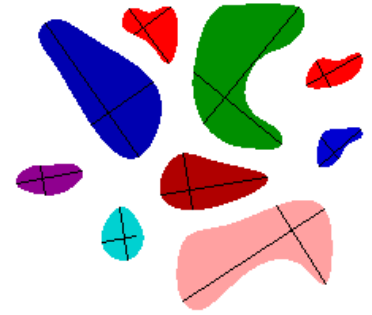
(c) Primeira implementação
(Limiar = 0.1)



(d) Primeira implementação
(Limiar = 0.05)



(e) Primeira implementação
(Limiar = 0.001)



(f) Segunda implementação

Figura 1 – Eixos maior e menor detectados na imagem `objetos3.png` pelo primeiro método. Note que valores altos de limiar na primeira implementação tendem a produzir eixos menores quase coincidentes com os eixos maiores, e valores baixos podem não produzir eixos menores.

A segunda solução utiliza um vetor diretor perpendicular à reta do eixo maior para traçar o segmento de reta “candidato” a ser o eixo menor. Assim, obteve-se a garantia de que todas as retas encontradas por esse método são perpendiculares ao eixo maior, sendo necessário apenas iterar sobre os pontos desse para encontrar a maior reta. Um exemplo visual pode ser observado na Figura 2. Note, na Figura, que o método escolheu uma reta levemente rotacionada em relação à horizontal para o eixo maior, apesar de se saber que o eixo maior da elipse representada é horizontal. Isso se dá devido à natureza discreta da representação da forma, conferindo a mais de um par de pontos da elipse a mesma distância máxima. Contudo, a excentricidade calculada para o objeto, de 0.864, é suficientemente próxima do valor real, igual a 0.866.

Essa solução, além de sempre produzir um resultado - o que não é garantido na primeira, pois o produto vetorial pode ser superior ao limiar para todo par de pontos -, é computacionalmente mais barata, uma vez que menos possibilidades de reta precisam ser checadas.

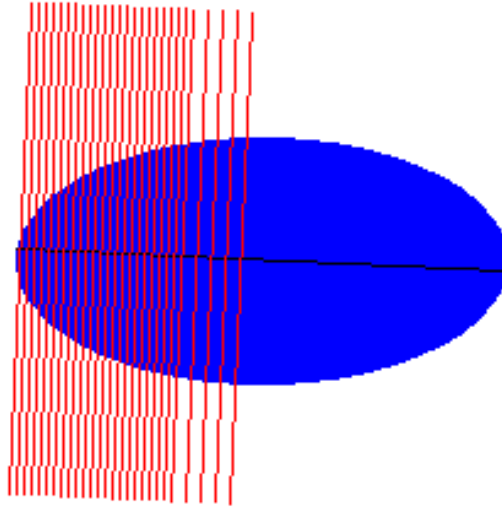


Figura 2 – Representação visual do método iterativo para encontrar o eixo menor na imagem `ellipse.png`. A reta associada ao eixo maior está representada em preto, enquanto as maiores retas encontradas para o eixo menor durante a iteração estão representadas em vermelho. A reta escolhida pelo algoritmo foi aquela mais à direita.

O segundo método, que envolve encontrar a elipse que mais se adéqua ao formato do objeto, foi implementado por meio da função `fitEllipse` da biblioteca *OpenCV*.

Foi possível observar, empiricamente, que os resultados obtidos pelos métodos citados são próximos para a maioria dos formatos presentes nas imagens testadas.

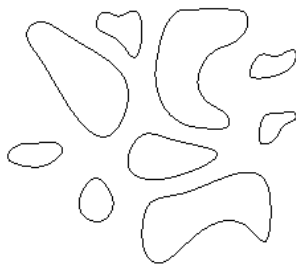
Por fim, após encontrar o comprimentos dos eixos maior e menor, o cálculo da excentricidade é realizado conforme a Equação 4. Os valores obtido por meio de ambos os métodos citados é apresentado após a execução do programa, na mesma ordem em que foram apresentadas nesta seção.

3.3 Resultados

Alguns dos resultados obtidos podem ser observados nas Figuras 3 e 4.



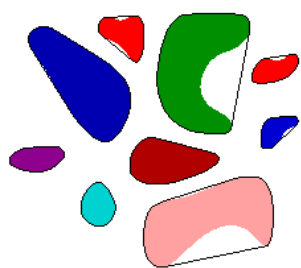
(a) Imagem binária



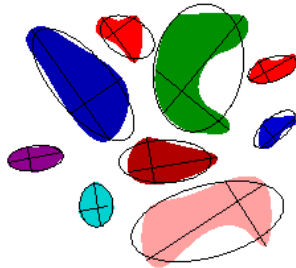
(b) Bordas dos objetos



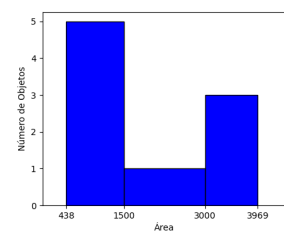
(c) Rótulos dos objetos



(d) Fechos convexos

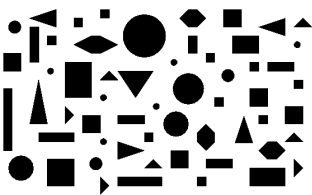


(e) Eixos maior e menor, e elipse ajustada

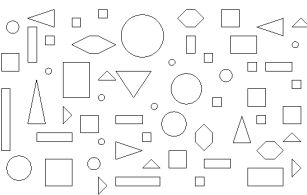


(f) Histograma de áreas

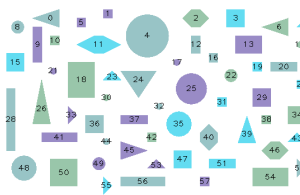
Figura 3 – Resultados gerados durante o processamento da imagem objetos3.png.



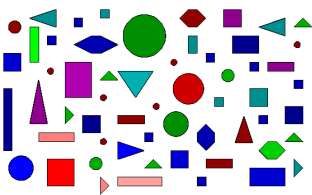
(a) Imagem binária



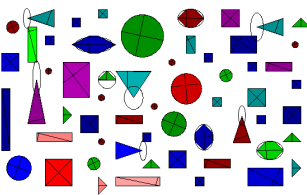
(b) Bordas dos objetos



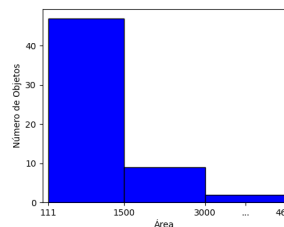
(c) Rótulos dos objetos



(d) Fechos convexos



(e) Eixos maior e menor, e elipse ajustada



(f) Histograma de áreas

Figura 4 – Resultados gerados durante o processamento da imagem objetos2.png.

3.4 Limitações

Destarte, devido ao processamento sequencial dos objetos e da complexidade elevada de algumas das operações realizadas, o tempo de execução tende a aumentar com o número de objetos na imagem.

Além disso, algumas formas podem apresentar valores de excentricidade distantes da realidade ou ausentes pelo segundo método (ajuste de elipse), uma vez que uma de suas etapas requer que o contorno apresente ao menos 5 pontos. Assim, como se observa na Figura 4, alguns dos triângulos possuem elipses que não descrevem corretamente a forma, ao passo que não é ajustada uma elipse aos quadrados. Nesses casos, o cálculo da excentricidade preferido é aquele produzido pelo primeiro método (eixos maior e menor internos ao objeto).

3.5 Conclusão

Foram implementadas, nesta atividade, diferentes estratégias para o cálculo de medidas geométricas sobre objetos em imagens binárias: área, perímetro, solidez e excentricidade. As três primeiras apresentaram cálculo direto ou com suporte robusto por bibliotecas existentes, enquanto a excentricidade exigiu abordagens mais elaboradas.

A área foi obtida pela simples contagem dos pixels rotulados com o valor correspondente ao objeto, e o perímetro, a partir do comprimento do contorno extraído. A solidez foi calculada por meio da razão entre a área do objeto e a área de seu fecho convexo, fornecendo uma medida da densidade da forma.

Para o cálculo da excentricidade, foram explorados dois métodos: o primeiro, baseado na iteração entre pares de pontos do contorno para encontrar os eixos maior e menor internos ao objeto, e o segundo, por meio do ajuste de uma elipse com a função `fitEllipse`. Ambos os métodos mostraram-se eficazes na maioria dos casos, com o primeiro sendo preferível em situações onde o ajuste de elipse falha (quando o objeto possui poucos pontos de contorno).

4 Transformações Geométricas

4.1 Introdução

Transformações geométricas são operações que permitem o mapeamento entre posições espaciais de pixels de uma imagem de referência e de uma imagem modificada a partir dela. Essas transformações consistem em duas operações: uma transformação espacial, que define como os pixels devem ser reajustados na nova imagem, e uma interpolação de intensidade, por meio da qual são atribuídos os valores de intensidade aos seus pixels.

No mapeamento direto, uma transformação espacial $\mathbf{T}$ que mapeia um ponto $\mathbf{P}$ da imagem original para o ponto $\mathbf{P}'$ na imagem modificada pode ser expressa como:

$$\mathbf{P}' = \mathbf{T} \mathbf{P} \quad (5)$$

Já no mapeamento indireto, a transformação inversa, $\mathbf{T}^{-1}$, pode ser representada como:

$$\mathbf{P} = \mathbf{T}^{-1} \mathbf{P}' \quad (6)$$

Essa, diferente da transformação original, garante que todos os pontos da nova imagem sejam mapeados para algum ponto da imagem original.

As coordenadas homogêneas permitem que as transformações geométricas sejam realizadas a partir de multiplicações matriciais; nestas, uma nova componente, W , é adicionada ao sistema de coordenadas cartesiano, de modo que um ponto (x, y, z) passa a ser representado como (wx, wy, wz, w) em coordenadas homogêneas. Essa substituição permite que as transformações geométricas sejam realizadas a partir de uma simples multiplicação matricial; assim, uma sequência de transformações $\mathbf{T}_1, \mathbf{T}_2, \dots, \mathbf{T}_n$ aplicadas a uma imagem, cujas respectivas matrizes homogêneas são $M_1, M_2, \dots, M_n$, pode ser substituída por uma única multiplicação matricial da imagem pela matriz M , dada por:

$$M = \prod_{i=1}^n M_i \quad (7)$$

A interpolação de intensidade, segunda etapa do processo de transformações geométricas, pode ser efetuada por meio de diversos métodos. Dentre eles, os aqui implementados e discutidos são: interpolação pelo vizinho mais próximo; bilinear; bicúbica e por Polinômios de Lagrange.

4.2 Metodologia

4.2.1 Transformações espaciais

As transformações espaciais de translação com deslocamentos t_x e t_y ; de rotação por um ângulo de α em sentido horário, e de escala por um fator s podem ser representadas, em suas formas matriciais, por $\mathbf{T}$, $\mathbf{R}_\alpha$ e $\mathbf{S}$, respectivamente, de modo que:

$$\mathbf{T} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}, \mathbf{R}_\alpha = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ e } \mathbf{S} = \begin{bmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (8)$$

Para que todos os pixels da imagem original, de dimensões $M \times N$, sejam mapeados após uma rotação de α graus, é necessário que a imagem transformada tenha dimensões $M \cos \alpha + N \sin \alpha \times M \sin \alpha + N \cos \alpha$. Além disso, uma relação mais trivial é que, para que o mesmo seja verdade após uma transformação de escala por um fator s , a imagem deve ter dimensões $sM \times sN$. Portanto, após a aplicação sequencial dessas transformações, a imagem gerada deve apresentar dimensões $M' \times N' = s(M \cos \alpha + N \sin \alpha) \times s(M \sin \alpha + N \cos \alpha)$, a fim de que todos os pixels sejam mapeados.

Ainda, uma vez que a rotação decorrente da aplicação de $\mathbf{R}_\alpha$ é realizada em relação à origem, é necessário transladar a imagem para que seu centro esteja localizado em $(M/2, N/2)$, i.e., aplicar uma translação com $t_x = -M/2$ e $t_y = -N/2$. Com isso, a rotação passa a ser aplicada a partir do centro da imagem; porém, é necessário retorná-la à origem do plano cartesiano após as transformações, resultado que pode ser obtido por uma translação com $t_x = -M'/2$ e $t_y = -N'/2$.

4.2.2 Forma de mapeamento

O método de mapeamento utilizado foi o indireto, uma vez que o mapeamento direto gera uma grade de pixels não mapeados na imagem resultante da transformação, como pode ser visto na Figura 5.

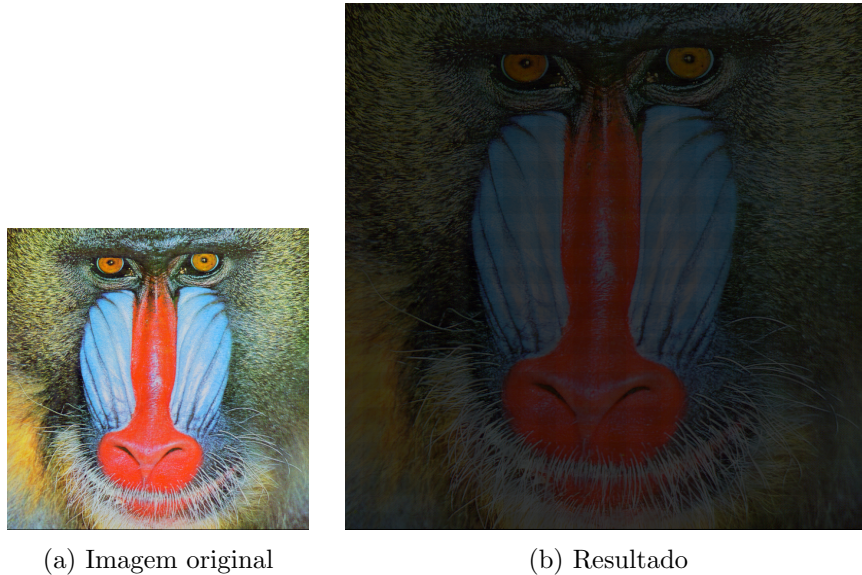


Figura 5 – Resultado da execução da interpolação de intensidade pelo vizinho mais próximo, após transformação de escala por um fator de 2.0, utilizando mapeamento direto. Pixels aos quais não foi atribuído um valor apresentam a cor preta, o que gera um efeito visual de escurecimento da imagem original.

4.2.3 Métodos de interpolação

Uma vez que os métodos de interpolação verificam uma vizinhança de até dois pixels ao redor de um pixel central, foi realizado um padding com replicação de borda na imagem de

entrada; devido a isso, todos os pixels da imagem resultante possuem um mapeamento na imagem de entrada.

Os detalhes de implementação dos métodos seguiram a especificação, e a diferença nos resultados gerados serão discutidos na próxima seção.

4.3 Resultados

Um resultado da aplicação do método pode ser observado na Figura 6. É possível notar que, na interpolação por vizinho mais próximo, as regiões de borda do texto apresentaram maior serrilhamento. Já nas interpolações bilinear e bicúbica, houve borramento do texto, havendo maior serrilhamento na primeira. Já na interpolação por Polinômios de Lagrange, os fenômenos de serrilhamento e borramento são menos perceptíveis.

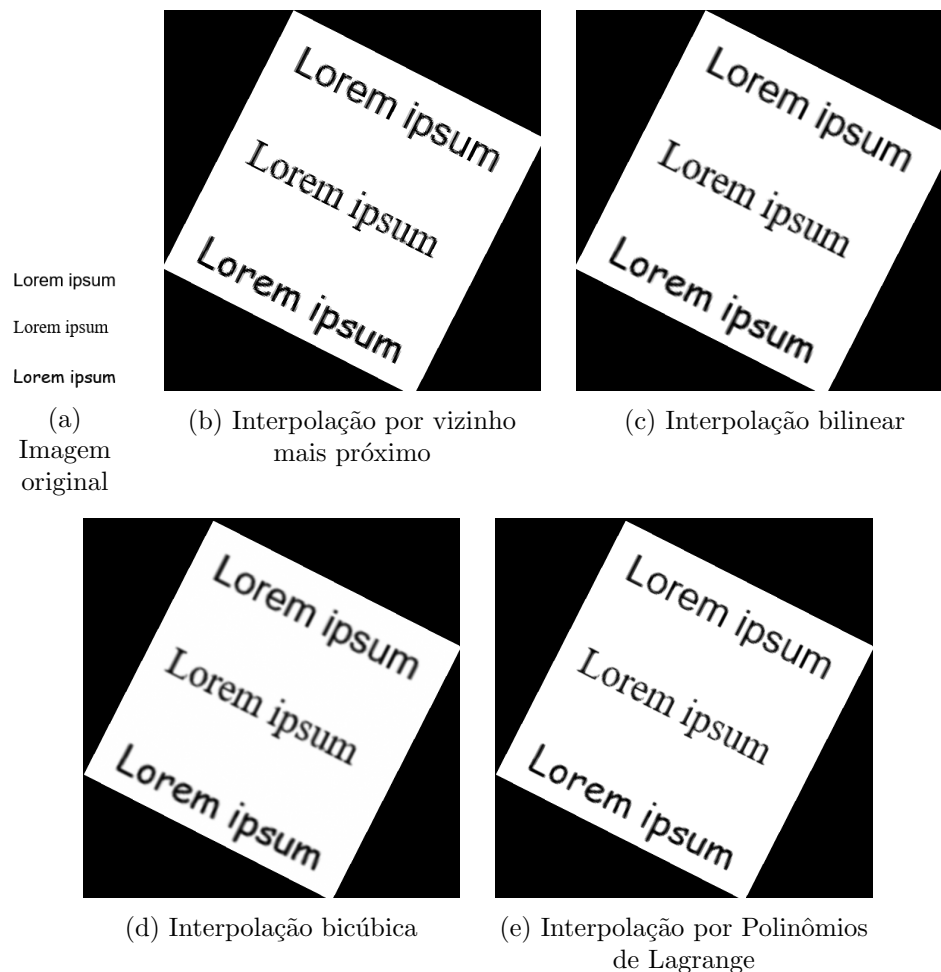


Figura 6 – Resultados gerados a partir da imagem `lorem_ipsum.png`, com 27° de rotação e fator de escala igual a 3.0.

Outros resultados pode ser observados na Figura 7. Embora os efeitos citados anteriormente sejam menos perceptíveis devido à natureza complexa da imagem, nota-se que o resultado gerado pela interpolação por Polinômios de Lagrange apresenta maior claridade.

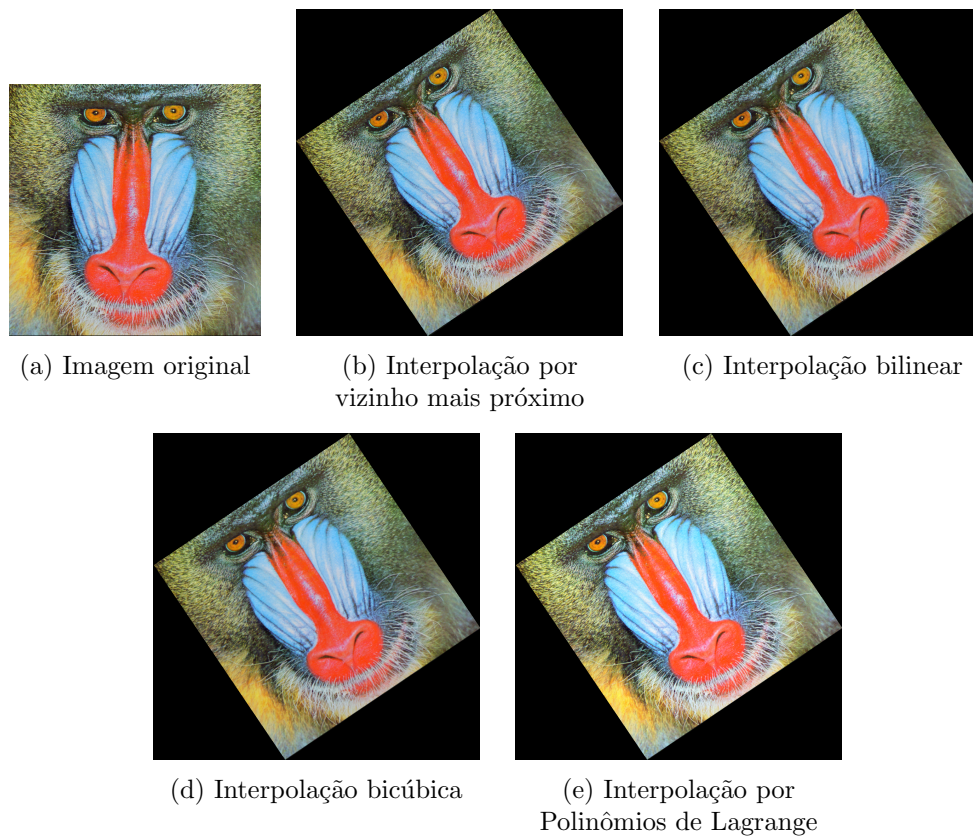


Figura 7 – Resultados gerados a partir da imagem `baboon_colorida.png`, com -34° de rotação e fator de escala igual a 1.3.

4.4 Limitações

O tempo esperado de execução do algoritmo aumenta quadraticamente com a escala, uma vez que, em todos os métodos de interpolação, é realizada uma iteração sobre todos os pontos da imagem de saída. Assim, para imagens muito grandes ou quando utilizando fator de escala elevado, é recomendada a definição de dimensões menores através do parâmetro `-D`, a fim de diminuir as iterações executados pelo algoritmo. Além disso, ao utilizar métodos mais simples (interpolações por vizinhos mais próximos e bilinear), as transformações tendem a ser processadas mais rapidamente; assim, deve ser feito um ponderamento entre qual possui maior importância: o tempo de execução ou a qualidade do resultado.

4.5 Conclusão

Foram implementadas, nesta atividade, transformações geométricas sobre imagens digitais por meio do uso de coordenadas homogêneas, permitindo a composição de múltiplas operações — translação, rotação e escala — em uma única multiplicação matricial. As transformações foram aplicadas utilizando o mapeamento indireto, que garante que todos os pixels da imagem de saída sejam devidamente mapeados para a imagem original, evitando lacunas.

Foram exploradas, também, diferentes técnicas de interpolação de intensidade: vizinho

mais próximo, bilinear, bicúbica e por Polinômios de Lagrange, cada uma com características próprias em termos de qualidade visual e custo computacional. Observou-se que, embora o vizinho mais próximo seja mais simples e rápido, ele introduz serrilhamento perceptível nas bordas. As demais técnicas resultam em imagens mais suaves, com destaque para a interpolação por Polinômios de Lagrange, que apresentou o melhor compromisso entre preservação de bordas e suavidade da imagem, apesar de tornar mais clara a imagem.

Por fim, notou-se que o tempo de execução do algoritmo está diretamente relacionado ao tamanho da imagem de saída. Assim, em aplicações com restrições de desempenho, é recomendada a limitação das dimensões da imagem, de forma a reduzir o número de iterações realizadas durante o processo.